DD2424 - Recurrent Neural Networks for Counteraction Recommendation in Air Defense Scenarios

William Frid

Spring 2026

1 Abstract

This report investigates whether it is possible to use a recurrent neural network (RNN or LSTM) in order to predict future threat levels in an air defense scenario based on simulation data. The problem is formulated as a supervised prediction task where the model receives a flattened state of the simulation including selected attributes of existing objects assuming perfect sensory coverage of the battlefield. These predictions are used to rank possible counteractions from the defending side. To find the optimal model, multiple searches are carried out, mainly using a custom written object-oriented framework that explored parameters such as sequence lengths, prediction horizons, architectures, and so on. The final results showed that it is possible to train a model that outperforms pure random counteraction selection (baseline of 0.15 win ratio) with the best performing model reaching up to 0.53 win ratio. However, the experiments also show that a low test loss in terms of Mean Squared Error (MSE) does not directly translate to a higher win ratio. Overall, the results indicated that recurrent models for counteraction recommendation are feasible and could potentially be used for decision making, but further work is needed.

2 Introduction

As warfare changes drastically due to modern technology, such as drones and Low Earth Orbit (LEO) satellites, the humans in command must also improve their decision-making capabilities. [1] [2]

[1] Council of the European Union. Space: the new battleground of modern warfare. (2025). Complete URL: https://www.consilium.europa.eu/media/yb5bkyey/2024_1015_art_space_11feb.pdf (2026-04-15).

[2] Kunertova, Dominika. The war in Ukraine shows that the game-changing effect of drones depends on the game. (Taylor & Francis, 2023). https://doi.org/10.1080/00963402.2023.2178180 (Accessed 15 April 2026).

One way of doing so is through calculated counteractions that could act as suggestions in stressed situations while keeping the human in control. This is something that is already being explored on multiple fronts, and was explored once more in this project.[3]

This project was however carried out without any domain expert, meaning behavior, attributes, and scenarios are highly questionable in terms of quality and realism. This does however not invalidate the results, but does decrease the validity and representability. Also note that the simulation data used for this project assumes perfect sensory cover for the both attacking, and defending side. Thus it is a question of decision, under perfect conditions.

2.1 Problem Formulation

The goal with this report is to answer the question "Can an RNN/LSTM model be trained using generated air defense scenarios well enough to provide useful recommendations?". Furthermore, as such computations must be done effectively, a second question to be answered would be "How must the data be used in terms of sequence lengths, prediction horizon, and so on, in order to generate high-quality recommendations?".

This project also covers in addition a sub-question, namely "How can one utilize AMD's ROCm as a substitution to Nvidia's CUDA cores for matrix operations?". A question that arose during the project due to limitations in terms of resources.

3 Methodology

3.1 Dataset Generation & Processing

For this project, multiple datasets were generated through a pre-existing simulation created as part of the SAAB's Smart Battle Command Hackathon 2026 (as seen in Figure 1). Each dataset records every 25th tick/state (value explained in later paragraph), where the defending side relied on reasonable random logic (meaning it was restricted to plausible counteractions), and the attacking side on hard-coded logic. These actions were carried out every 100th tick (see Figure 2). Each simulation was run and recorded up to victory or defeat for either side, with a maximum of 200.000 ticks. If a simulation were to reach 200.000 ticks, it would count as a win for the defending side as it withstood the attacker.

[3]Petr Gallus, Petr Františ. Command-and-Control System Analysis and Delineation of Possible Areas for Machine Learning. (University of Defence, 2025). https://aimt.cz/index.php/aimt/article/view/1948 (2026-04-15).

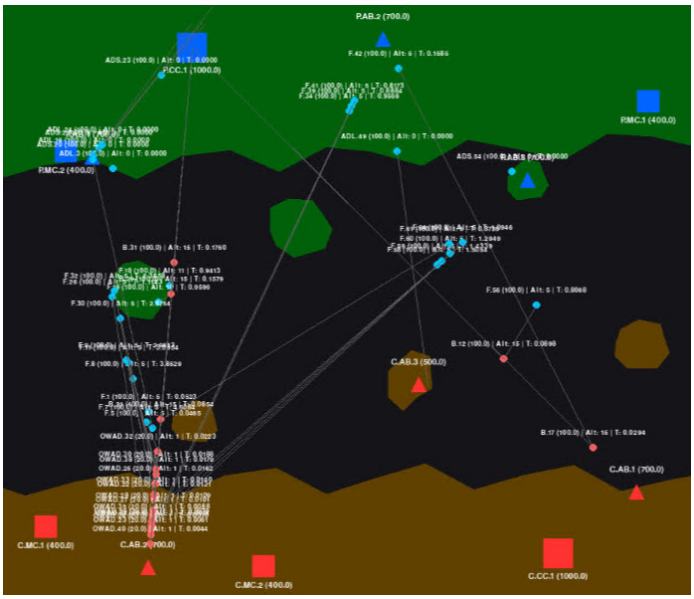


Figure 1: Simulation showing defending (north) and attacking (south) sides. Small square represents major cities, large square represents capital cities, triangles represents air bases, and circles represents weapons.

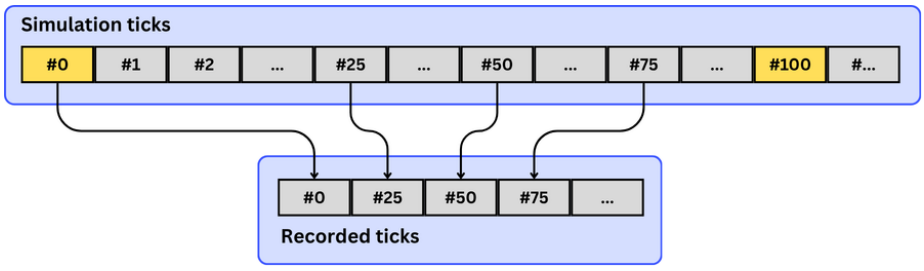


Figure 2: Actions are carried out every 100th tick, which is represented with yellow cells. Only every 25th state was recorded.

During the recording of data from the simulation, only the most valuable attributes were stored including for instance weapon type, location (X, Y , Z), health, target object/position, and threat level. The threat level was calculated using Equation 1.

$$\frac{damage}{target_health} * \frac{starting_distance_to_target - distance_to_target}{starting_distance_to_target} \quad (1)$$

Furthermore, player objects (air bases and cities), as well as total threat levels were included. In total, this resulted in a total of 2.092 values.

This data was then processed through padding (see Figure 3), global normalization, and flattening (see Figure 4) to reach a consistent vectorized state of the simulation which later could be interpreted by the neural network.

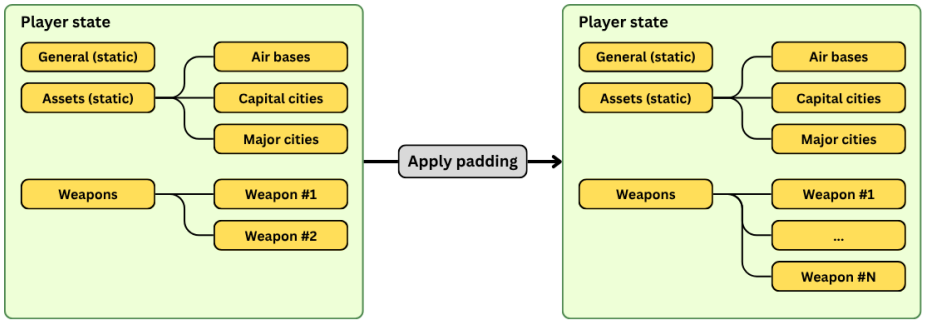


Figure 3: Padding was applied to player state to compensate for differing weapon list lengths. This was done through zero padding to reach a pre-determined size.

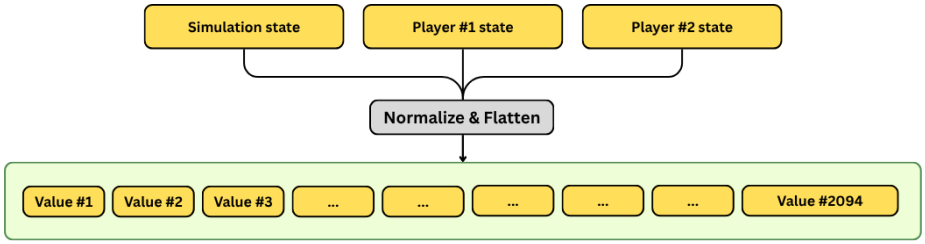


Figure 4: Global normalization was applied to the data using constants such as map size, and maximum allowed values for certain attributes. This was then flatten to fit in a single vector.

The resulting machine learning task was formulated as a supervised sequence prediction task. Each input was made out of a sequence of flattened simulation states, while the targets were only the two attributes representing total attacker, and defender threats after a decided *look_ahead* horizon. The models were therefore trained to minimize the MSE of these future threat values, and these were later used for counteraction decisions.

In order to optimize the tests, an assumption had to be made regarding suitable ticks between recording state. A short test using the Tensorflow RNN Model (see Section 4.1) was run 5 times with different datasets made out of 240 simulations recorded at different intervals. Mean performances were calculated from these results, showing reasonable results for each of the relevant options (see Figure 5). In the end, 25 was picked as a trade-off between performance, storage, and speed.

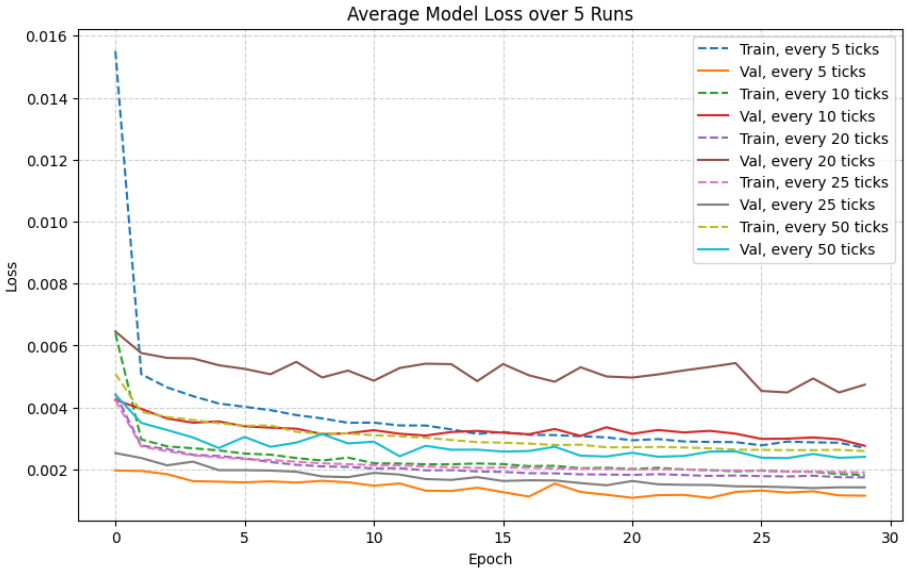


Figure 5: Plot showing the training process for different state saving steps. Note how most cases have similar results, with the only outlier being every 20th step.

4 Model Architectures

4.1 Tensorflow RNN Model

In order to explore the effects of different dataset generation methods and usage, a simple Tensorflow based RNN model was set up with the following structure:

1. SimpleRNN(32 units, Dropout(0.2))
2. Dense(32 units, Relu)
3. Dropout(0.3)
4. Dense(2 units, Linear)

This model will be referred to as the "Tensorflow RNN Model" throughout this report.

4.2 Tensorflow LSTM Model

To further explore effects of the dataset and getting an understanding for the effect of swapping from RNN layers to LSTM layers, another model was set up using Tensorflow. This model had the structure:

1. LSTM(64 units, Dropout(0.2))

2. LSTM(32 units, Dropout(0.2))
3. Dense(32 units, Relu)
4. Dropout(0.3)
5. Dense(2 units, Linear)

This model will be referred to as the "Tensorflow LSTM Model" throughout this report.

4.3 Custom Neural Network Implementation

For the custom neural network implementation, an object oriented approach was used. The framework was built to support the basic functionalities of training, validation and testing. As well as Adam Optimizer, L2-Regularization, activation function selection, and data generators (for reducing memory usage). It lacked typical dropout which was cut out due to time limitations. Note that due to the absence of dropout, these models could not fairly be compared the two previously mentioned Tensorflow based models. This framework was first built using NumPy, but later migrated to CuPy (see Section 5.2).

4.3.1 Base Custom LSTM Model

During the development of the framework, a simple model had to be used for debugging. This model was also used for minor testing such as the one under Section 6.3. The model used for this had the structure:

1. LSTM(100 units)
2. Dense(100 units, Relu)
3. Dense(2 units, Linear)

5 Technical Implementation & Optimization

5.1 Hardware & Environment

The test run for this report were run on a Ubuntu system based on an AMD Ryzen 7 5800XT, AMD RX 9070, 32GB of 3GHz memory, and NVMe PCIe M.2 storage for dataset.

As the setup was AMD based, it required a Docker container built for AMD's ROCm to support faster computations (for machine learning tasks).[4] This was modified to also support the experimental library CuPy which was implemented instead of NumPy as part of the project.[5] This means that the Docker Container

[4] Advanced Micro Devices. AMD ROCm™ Software. (2026). https://www.amd.com/en/products/software/rocm.html (Accessed 2026-05-20).

[5] Preferred Networks, Inc. and Preferred Infrastructure, Inc. CuPy: NumPy & SciPy for GPU. https://cupy.dev/ (Accessed 2026-05-20).

was extended with the additional CuPy package (version 7.0), which required installation through the package manager pip that was set up to target correct GPU architecture.

5.2 Framework Migration

To migrate from NumPy to CuPy is a straight forward process once the setup is working. What needs to be done it to replace each non-data-reading process NumPy action with CuPy, as the reading of data is CPU bound. Meaning once the data is loaded, convert the data with a Python line such as $X = cp.asarray(X)$, and by doing so, we move the data from memory to VRAM (video memory connected to GPU). Then, for computations CuPy is used which then uses the data stored on the VRAM. Resulting in a theoretical speedup. *Note however that when saving a model as a .npy file, it must be converted back to NumPy, and vise versa when loading a model from a .npy file.*

For improved stability, manual garbage collection had to be added for the GPU' VRAM specifically through the CuPy library. This was added after each individual models training (in loops) as multiple models were explored through different search approaches.

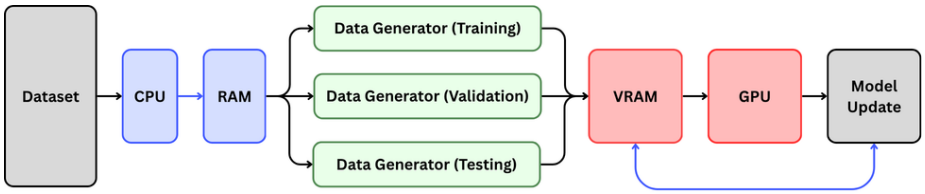


Figure 6: By using CuPy and moving data to VRAM and performing computations on GPU, the process can be greatly speed up. Combining this with data generators (pipelines), and we get a robust system which won't suffocate on large dataset populating the systems memory.

Due to the limited stability of a GPU in terms of memory management performing multi-tasking (being called from multiple processes to perform predictions using a ML model), the implementation was adjusted to support both NumPy and CuPy. This would allow the developer to pick the most suitable approach for each individual model, where NumPy would be running $0.8 * num_of_cpu_cores$ processes when collecting simulation data, and CuPy would be limiting it to 2 processes. Note however that CuPy was always used for the training process. Furthermore, the memory allocation issue on the GPU can be worked around through synchronized calls.

5.3 Data Generators (Pipelines)

Due to the dataset's large size data generators (or pipelines) were added to handle batch loading of data from storage in order to prevent crashing runs

due to memory limitations. For each run, three new unique generators were created, one for training, validation, and testing. While this approach does add overhead computations (loading batch by batch), it does allow for more stability and reruns without risking the Python kernel crashing. Note that this does require proper cache handling through garbage collection.

6 Results & Analysis

6.1 Migrating From NumPy to CuPy

The results of migrating from NumPy to CuPy showed a great speed improvement. The test was run over 5 times (form where a mean was calculated) for 10 training epochs on a dataset of 144 training samples, 48 validation samples, using the Tensorflow RNN Model (see Section 4.1). The results are seen below, with the CuPy implementation giving a time decrease of 503 seconds, or 87% in relative decrease.

- NumPy: 579s
- CuPy: 76s

It was however discovered that when calling the simulation for testing a model "in action", that using the GPU to predict could in certain cases result in slower speeds because of the data transferred from RAM to VRAM overhead. This combined with the previously mentioned stability issue (unless you implement synchronous calls) shows the importance of analyzing the problem before converting from CPU to GPU based calculations.

6.2 Tensorflow Baselines

6.2.1 Sequence Length

From the decision to use data recorded every 25th tick, a second test was run to explore the effect of sequence lengths. These were explored with certain multiplier steps, meaning exploration was done over multiples of 4. Looking at Figure 7, most options are relatively close to one-another, except for one outlier (sequence length of 16).

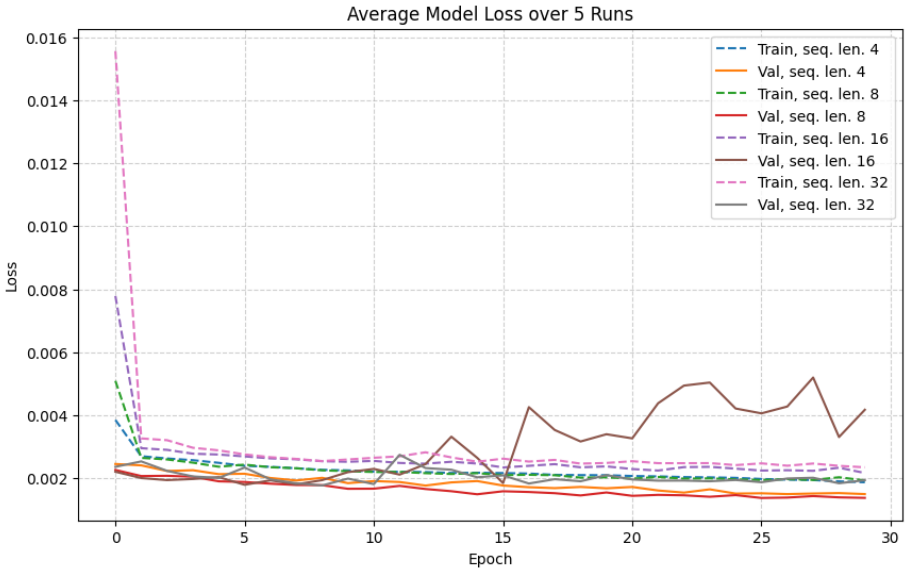


Figure 7: Effect of different sequence lengths using the Tensorflow RNN model.

6.2.2 Look Ahead

To determine the effect of increasing the look ahead steps, another test was run, exploring the performance of looking ahead $X * 4$ steps (see Table 1). These results can be seen in Figure 8, in which it is clear that the predicting further into the future, leads to higher loss. These did however not lead to a final decision of the set *look_ahead* value, as it has to be tested in the simulation.

look_ahead	**Sequence steps**	**Time**
1	4	100s
3	12	300s = 5min
9	36	15 min
18	72	30 min
27	108	45 min
36	144	1 h
72	288	2 h
108	432	3 h
144	576	4 h
180	720	5 h
216	864	6 h

Table 1: The look ahead steps converted into sequence steps and time.

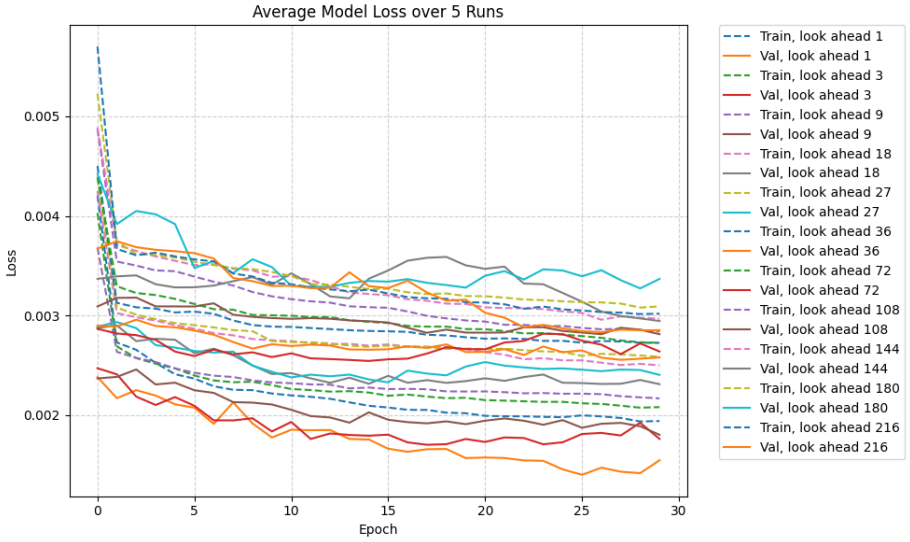


Figure 8: Effect of different *look_ahead* values using the Tensorflow RNN model.

6.2.3 Look Ahead & Sequence Length

By comparing *look_ahead* and sequence length directly, it can be seen in Figure 9 that the models behaves as one would expect from the results seen in sections 6.2.1 and 6.2.2. While somewhat longer sequence length does improve the performance slightly, increasing the prediction horizon (increasing *look_ahead*) always has a negative impact on the MSE.

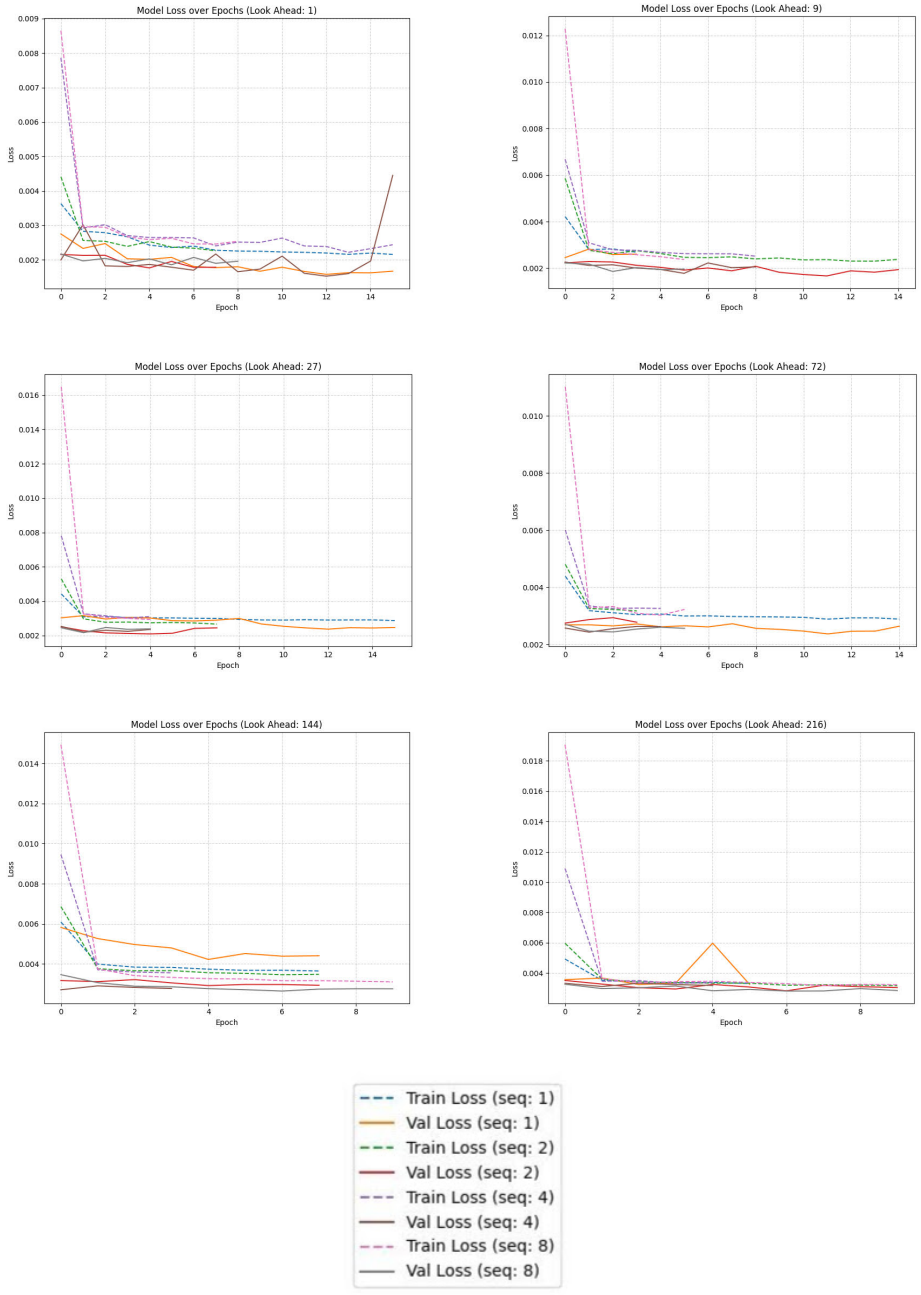


Figure 9: Training results from exploring different *look_ahead* values against sequence lengths.

6.2.4 Sequence Steps

To determine how much the data could be overlapping when sampling from the training data through the data generator, a test was run with the different step sizes of 1, 2, 3, and 4. Where 1 would mean maximum overlap giving the largest amount of data. through risking that the model memorizes more than learning. Based on the results seen in Figure 10, the best performing setting is a smaller amount of overlap, with the outlier of sequence step 3 which is very close to sequence step 4.



Figure 10: Plot showing the affects of different steps in data input in terms of ticks. 4 steps means zero overlap as the test used 25ticks between each recorded state.

6.3 Decision Metric & Baseline

By comparing the simulation win rate over a 100 runs using the Base Custom LSTM Model (see Section 4.3.1), it was concluded that the best option to select actions based on the total attacker and defender scores, was to use the ratio seen in Equation 2. Note that this model was trained using the best values found during the Tensorflow baseline tests (Section 6.2) over 10 epochs. All results from this test can be seen in Table 2.

$$ratio = \frac{total_defender_threat}{total_attacker_threat} \tag{2}$$

Model	Strategy	Win ratio
None	Random	0.15
LSTM	Minimize attacker threat	0.09
LSTM	Maximize defender threat	0.23
LSTM	Max. defender thr. & min. attacker thr.	0.35

Table 2: Action selection strategy.

The first row in this table showing the result of pure random decisions acts as this projects **baseline**. Meaning a result above 0.15 win ratio over 100 simulation is deemed a success in terms of trained model.

6.4 Wide Random Search

The wide search for suitable dataset usage and model architecture was carried out without any regularization, but with the larger dataset of 2.400 simulation runs (using a 0.6/0.2/0.2 split). This was a typical random search with some grid search behavior, exploring:

- LSTM versus RNN
- Amount of LSTM/RNN layers
- Amount of fully connected layers
- Units in each layer
- Sequence length
- Batch size

This algorithm was run 10 times for each of the *look_ahead* steps seen in Table 1 (resulting in 110 models). Form these results, the top performers in terms of MSE for each *look_ahead* was reloaded, and used for the simulation 100 times to calculate a winning ratio for the defending side. From this, three models (*look_ahead*) stood out. These can be seen in Table 3.

Batch size	Seq. len.	Lookahead	Step	Layer type	Activation	Units	Input dim.	Test loss	Win ratio
64	4	72	4	LSTM	–	16	2092	0.0042	0.40
				Fully connected	ReLU	256	16		
				Fully connected	ReLU	64	256		
				Fully connected	Linear	2	64		
64	16	144	4	Simple RNN	Tanh	16	2092	0.0058	0.36
				Fully connected	ReLU	16	16		
				Fully connected	ReLU	128	16		
				Fully connected	Linear	2	128		
32	8	180	4	LSTM	–	32	2092	0.0062	0.39
				Fully connected	ReLU	64	32		
				Fully connected	ReLU	32	64		
				Fully connected	Linear	2	32		

Table 3: Architecture overview of the top three models from the wide search.

Based on these results, we can conclude multiple things: First of all, **the training was successfully** as it managed to train models that beat the baseline of 0.15 win ratio. Secondly, all uses sequence steps of 4, meaning during training, there is no overlapping steps when selecting input data via the data generators. We also see that the top three models use sequence length of 4, 8, and 16, leaving longer sequences such as 32 and 64 out (see Table 7 for interpretation). This seems to be combined with a *look_ahead* of 72+ suggesting longer prediction leads to improved win ratio. The final data usage insight is that all three uses batch sizes of 32 and 64, excluding smaller like 16, and larger such as 128 and 256.

When it comes to architecture, the top two performing models (in terms of win ratio) both uses LSTM and not RNN. But all three models has three dense layers (including final one also acting as output layer).

6.5 Narrow Random Search

As a continuation of the search, a second more narrow random search was conducted. This search built on results found in the wide random search, and focused on exploring:

- Higher *look_ahead* values (72+)
- Batch sized 32 and 64
- Sequence length
- Amount of unit in LSTM layer (single layer with smaller amount of units)
- Amount fo fully connected layer
- Amount of units in each fully connected layer
- L2-Regularization in each layer

From this search, 50 models , and 5 models stood out, which can be seen in Table 4. However, both the test loss, and win ratio for each of them were significantly lower compared to the ones found during the wide search. Further more, none of the best performing models used any L2-regularization.

Batch size	Seq. len.	Lookahead	Step	Layer type	Activation	Units	Input dim.	λ	Test loss	Win ratio
64	4	72	4	LSTM	–	32	2092	0	0.0044	0.04
				Fully connected	ReLU	256	32	0		
				Fully connected	Linear	2	256	0		
64	16	144	4	LSTM	–	64	2092	0	0.0053	0.12
				Fully connected	ReLU	256	64	0		
				Fully connected	ReLU	32	256	0		
				Fully connected	ReLU	128	32	0		
				Fully connected	Linear	2	128	0		
64	8	180	4	LSTM	–	64	2092	0	0.0068	0.15
				Fully connected	ReLU	256	64	0		
				Fully connected	ReLU	1024	256	0		
				Fully connected	Linear	2	1024	0		
64	4	180	4	LSTM	–	16	2092	0	0.0078	0.22
				Fully connected	ReLU	1024	16	0		
				Fully connected	ReLU	64	1024	0		
				Fully connected	Linear	2	64	0		
64	8	180	4	LSTM	–	32	2092	0	0.0073	0.21
				Fully connected	ReLU	64	32	0		
				Fully connected	Linear	2	64	0		

Table 4: Architecture overview and test performance of the evaluated LSTM models, including batch size, sequence length, lookahead, step size, and L2 regularization strength λ.

6.6 Second Wide Search

Due to the fall in performance between wide random search and narrow random search, a third random search was run that acted as a combination of the previous two. This search was used to find 200 potential models, from where the top performing of each unique *look_ahead* was selected. These models were tested in the simulation, resulting in the result seen in Table 5.

Looking at the last model, it managed to reach a win ratio of 0.53, beating the previously best performing model with an improvement of +0.14 ratio. A model which in fact relied on RNN and not LSTM, with a total of 5 layers and again no regularization.

Batch size	Seq. len.	Lookahead	Step	Layer type	Activation	Units	Input dim.	λ	Test loss	Sim. score
64	4	72	4	Simple RNN	Tanh	64	2092	0	0.0055	0.22
				Simple RNN	Tanh	64	64	0		
				Fully connected	ReLU	64	64	0		
				Fully connected	Linear	2	64	0		
128	4	144	12	LSTM	–	64	2092	0	0.0055	0.42
				LSTM	–	16	64	0		
				Fully connected	ReLU	1024	16	0		
				Fully connected	ReLU	32	1024	0		
				Fully connected	Linear	2	32	0		
128	4	216	4	Simple RNN	Tanh	16	2092	0	0.0066	0.53
				Simple RNN	Tanh	32	16	0		
				Fully connected	ReLU	512	32	0		
				Fully connected	ReLU	256	512	0		
				Fully connected	Linear	2	256	0		

Table 5: Architecture overview and test performance of the final selected models, including batch size, sequence length, lookahead, step size, and L2 regularization strength λ.

6.7 Pushing the Performance

The final tests run explored the effect of increasing the amount of explored futures during the simulation. In Table 6, we can see that increasing this value

Random Position	Random Target	Win Ratio
5	10	0.53
10	20	0.56
20	40	0.46
40	80	0.47

Table 6: Results of predicting outcome of different amounts of random actions.

slightly (doubling base value) did improve the performance, but increasing it further did not.

Lastly, the top performing model was trained once again, this time with a dataset of size 7.200 simulations (again with 0.6, 0.2, 0.2 split). The training progress can be seen in Figure 11 in which we see that the model converges quickly in terms of MSE and does not show clear signs of overfitting as the training and validation loss remains close after epoch 6. However, the flat curve indicates that the model have reached a performance plateau. After 12 epochs it stops due to early stopping implementation.

The observed peak can be due to pure randomness, too high learning rate, error in data generator pipeline or other factors. Though the end result is stable, indicating this is likely no greater issue.

After having trained this model further and run the simulation, a win ratio of 0.4 was reached. Meaning lower performance compared to before training it further.

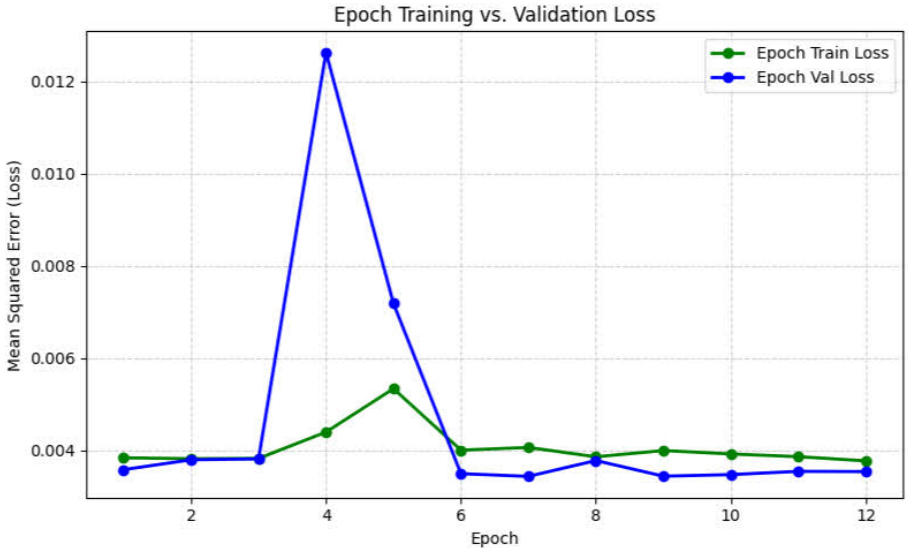


Figure 11: Further training of best performing model using a larger dataset of 7.200 simulations.

7 Discussion & Conclusions

Based on the results found, it is clear that implementing **CuPy** over NumPy to utilize the GPU's full potential is a viable option for training a RNN or LSTM. Implementing either of these from scratch does however require major optimization in terms of resource utilization (storage, memory, and computational computational power). To take such an implementation further, one would need to add a synchronization module that would build up a batch of predictions to perform at a time from multiple simulation/processes. By doing so, the GPU would be fully utilized and not locked to a few tasks at a time.

For a model to give recommendations regarding counteractions in an air defense scenario, this simulation suggests that it is more important to make out a threat against the attacking side, rather than minimizing the threat the attacking side make up. However, the better **decision metric** found was a combination in the form of a ratio of the two outperforms both. This could potentially be further improved through weighting the two values (one weight for numerator, and one for denominator).

From the **Tensorflow baseline tests**, exploring sequence length, steps, and $look_ahead$, results suggesting longer sequence lengths did not lead to any clear improvement compared to shorter sequence lengths. It also showed that less overlap of training data is to be preferred, and that the accuracy of predictions further into the future becomes less reliable. These findings were however found using a RNN model, meaning they could potentially differ somewhat if tested with an LSTM model.

The results from the Tensorflow baseline model are projected to all three random searches. Although the **first random search** (which excluded the search for L2-regularization strength) showed promising results in terms of win ratio up to 0.40, the **narrow random search** (with adjusted search space and added search for L2-regularization strength) failed, only searching for a win ratio up to 0.22. Then, through the second wide search (a combination of the former two), the results were restored, reaching 0.53.

During the runs of simulation tests, it was quickly clear that the test loss did not directly depict the resulting win ratio. As the highest performing model in terms of win ratio would be the one that had higher $look_ahead$, whereas the model with lowest test loss would be one with a lower $look_ahead$ value. To further investigate this behavior, one should take multiple trained models of the same $look_ahead$, and compare their test loss to their corresponding simulation win ratios.

It is important to keep in mind that the test loss (MSE) does not reflect the expected simulation win ratio because of multiple reasons; large problem space, limited data set, and simulation nature to mention a few. The latter one being the main reason as certain scenarios could potentially be impossible to defend against given the simulation nature. This could be a single all-or-nothing attack or chaotic random target attacks.

To further investigate this challenge, the following should be done:

- Run longer and more sophisticated random search, including dropout-layers
- Run longer simulation tests
- Explore effect of adding transformers
- Explore the effect of other attribute mixtures in each data point
- Explore decision metric weights (see Section 6.3) through a grid-search
- Perform the suggested comparison between test loss and simulation win ratio between models with the same *look_ahead* value

Based on the found results, we see that it is possible to train a model to perform better than purely random action selection (baseline). Though this performance peaked at 0.53 (0.56 with more data and training), it still shows promising results with possibility of improvement. Even though this score is far below what one would prefer in a real-life application, one must keep into consideration that the project was carried out without a domain expert, no real data, and with limited resources (limiting tests, searches, and simulated data amounts). With this, **the answer to the first question** "Can an RNN/LSTM model be trained using generated air defense scenarios well enough to provide useful recommendations?", would be: Yes, the tests show promising results for further development and improvement possibilities.

For the second question: "How must the data be used in terms of sequence lengths, prediction horizon, and so on, in order to generate high-quality recommendations?", clear results were found for the architectures explored. Shorter sequence lengths combined with a sequence step matching the action interval appeared to perform best. This combined with a higher *look_ahead* value led to the best performance in terms of simulation win ratio, even though higher *look_ahead* generally resulted in worse MSE. In the end, it is a question of interpretation of the two.